Local Testing Guide: Deploying WHMCS with Elastic APM

This guide provides step-by-step instructions to build the updated WHMCS container image, deploy a local Elastic Stack (Elasticsearch, Kibana, APM Server) in your homelab cluster, and verify that telemetry flows correctly without performance degradation.

Step 1: Build the WHMCS Docker Image locally

First, compile the Docker image containing the updated `php.ini`, `entrypoint.sh`, and the new APM agent `v1.17.0` package.

Run this command from the root of the WHMCS repository (`/Users/gedeon/Dev/work/tcloud-whmcs`):

```
# Build the image using latest theme and addons
docker build \
  --build-arg THEME_TAG=latest \
  --build-arg ADDONS_TAG=latest \
  -t homelab/whmcs-app:optimize-whmcs .
```

Note: If testing on a multi-node Kubernetes cluster, push the built image to your local homelab registry, or load it into your local cluster nodes (e.g. `minikube image load homelab/whmcs-app:optimize-whmcs` or `k3d image import ...`).

Step 2: Deploy a Local APM & Elastic Stack

To receive telemetry, you need Elasticsearch, Kibana, and the Elastic APM Server running locally.

Option A: Kubernetes Manifests (Recommended for Homelab K8s)

Apply the following manifests in a namespace called `monitoring` (or similar) to deploy a lightweight single-node stack:

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: elasticsearch
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: elasticsearch
  template:
```

```yaml
    metadata:
      labels:
        app: elasticsearch
    spec:
      containers:
        - name: elasticsearch
          image: docker.elastic.co/elasticsearch/elasticsearch:7.17.9
          ports:
            - containerPort: 9200
          env:
            - name: discovery.type
              value: single-node
            - name: xpack.security.enabled
              value: "false"
---
apiVersion: v1
kind: Service
metadata:
  name: elasticsearch
  namespace: monitoring
spec:
  ports:
    - port: 9200
  selector:
    app: elasticsearch
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kibana
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kibana
  template:
    metadata:
      labels:
        app: kibana
    spec:
      containers:
        - name: kibana
          image: docker.elastic.co/kibana/kibana:7.17.9
          ports:
            - containerPort: 5601
          env:
            - name: ELASTICSEARCH_HOSTS
              value: "http://elasticsearch:9200"
---
apiVersion: v1
kind: Service
```

```yaml
metadata:
  name: kibana
  namespace: monitoring
spec:
  ports:
    - port: 5601
  selector:
    app: kibana
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apm-server
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apm-server
  template:
    metadata:
      labels:
        app: apm-server
    spec:
      containers:
        - name: apm-server
          image: docker.elastic.co/apm/apm-server:7.17.9
          ports:
            - containerPort: 8200
          env:
            - name: apm-server.host
              value: "0.0.0.0:8200"
            - name: apm-server.rum.enabled
              value: "true"
            - name: output.elasticsearch.hosts
              value: "['http://elasticsearch:9200']"
---
apiVersion: v1
kind: Service
metadata:
  name: apm-server
  namespace: monitoring
spec:
  ports:
    - port: 8200
  selector:
    app: apm-server
```

Step 3: Deploy a Local MySQL Database (with Persistent Storage)

Apply this manifest to deploy a single-node MySQL 5.7 database with a 5GB PersistentVolumeClaim (PVC) in the default namespace:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:5.7
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "root_password"
            - name: MYSQL_DATABASE
              value: "whmcs_db"
            - name: MYSQL_USER
              value: "whmcs"
            - name: MYSQL_PASSWORD
              value: "whmcs_password"
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
```

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: default
spec:
  ports:
    - port: 3306
  selector:
    app: mysql
```

Step 4: Deploy the WHMCS App Locally in K8s

Deploy the WHMCS application container pointing to the local APM server and the newly created local MySQL service:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: whmcs-local
  namespace: default
spec:
  replicas: 1
  selector:
    matchLabels:
      app: whmcs-local
  template:
    metadata:
      labels:
        app: whmcs-local
    spec:
      containers:
        - name: whmcs-app
          image: homelab/whmcs-app:optimize-whmcs
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
          env:
            - name: INSTRUMENT_ELASTIC_APM
              value: "True"
            - name: APM_SERVICE_NAME
              value: "local-whmcs"
            - name: APM_SERVER_URL
              value: "http://apm-server.monitoring.svc.cluster.local:8200"
            - name: DB_HOST
              value: "mysql"
            - name: DB_PORT
              value: "3306"
            - name: DB_USER
```

```
            value: "whmcs"
          - name: DB_PASSWORD
            value: "whmcs_password"
          - name: DB_NAME
            value: "whmcs_db"
          - name: WHMCS_LICENCE
            value: "YOUR_DEV_LICENSE"
```

Step 5: Verify Telemetry and Optimization in Kibana

1. **Access Kibana**: Port-forward to Kibana locally:

   ```
   kubectl port-forward svc/kibana -n monitoring 5601:5601
   ```

   Open `http://localhost:5601` in your browser.

2. **Generate Traffic**: Trigger requests on the local WHMCS deployment (e.g. click around the client area or administration panels).

3. **Check the APM Dashboard**:

   - In Kibana, go to **Observability** > **APM** > **Services**.
   - You should see `local-whmcs` active in the service list.
   - Click on the service to verify that transactions, trace waterfall graphs, and SQL queries are logged.

4. **Verify Memory Allocation and Asynchronous Offloading**:

   - Check that `USE_ZEND_ALLOC` is enabled (`USE_ZEND_ALLOC=1` by default) by accessing a simple `phpinfo.php` file inside the container, or verify that CPU utilization remains low.
   - In `phpinfo.php` (or running `php -i`), verify that `elastic_apm.async_backend_comm` displays as `true` (confirming that trace uploading is executing asynchronously in the background).